

Monitoramento e otimização de recursos em aplicativos Flutter

Beatriz Vocurca Frade

Universidade Federal de Minas Gerais
Engenharia de Sistemas
Orientador: Prof. Ramon Lacerda Marques
2026



Problema e motivação

- Flutter acelera o desenvolvimento multiplataforma, mas não elimina gargalos de desempenho.
- Apps modernos combinam listas dinâmicas, animações, rede, estado reativo e dispositivos muito diferentes.
- Problemas aparecem como *jank*, queda de FPS, alto uso de memória, chamadas de rede excessivas e menor duração de bateria.

Pergunta de pesquisa

Como tornar o diagnóstico de desempenho mais acessível, contínuo e acionável em Flutter?

Motivação prática

A primeira leitura do problema deve acontecer enquanto o aplicativo ainda está sendo construído.

Objetivos

Objetivo geral

Desenvolver e avaliar funcionalmente o pacote `collector_flutter`, uma biblioteca Dart/Flutter embarcada para coletar métricas, analisar sintomas e apresentar recomendações em tempo real.

- Coletar FPS, *jank*, memória, rede, CPU, bateria, eventos e *rebuids*.
- Organizar a solução em arquitetura modular.
- Exibir métricas e recomendações no próprio app.
- Gerar recomendações heurísticas por severidade.
- Exportar sessões em JSON e *frame timings* em CSV.
- Validar o comportamento em cenários controlados.

Base científica e lacuna

Energia

Consumo móvel se relaciona a CPU, rede, execução, memória e uso contínuo do dispositivo.

Performance móvel

A análise precisa observar múltiplos sinais, não apenas uma métrica isolada.

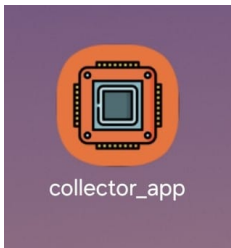
Diagnóstico

Sintomas técnicos precisam virar hipóteses de otimização compreensíveis.

A lacuna está entre medir e agir: ferramentas externas são fortes, mas a triagem inicial ainda tem atrito no fluxo diário de desenvolvimento.

Proposta: biblioteca desenvolvida

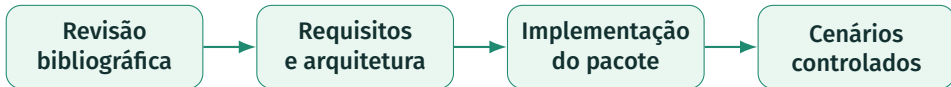
- O pacote `collector_flutter` é integrado à aplicação monitorada.
- A ferramenta proposta coleta métricas durante a execução do app.
- A análise heurística classifica sintomas e gera recomendações por severidade.
- O dashboard mostra sinais de desempenho sem exigir troca imediata para uma ferramenta externa.



Ideia central

Aproximar o primeiro diagnóstico do fluxo real de desenvolvimento Flutter.

Metodologia



Tipo de avaliação

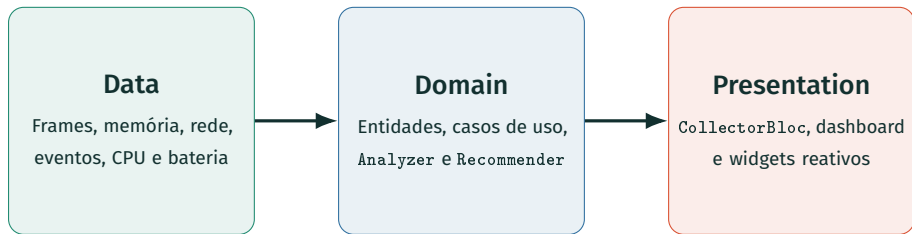
Validação funcional em app de exemplo, com observação de sinais induzidos por cenários controlados.

Critério de leitura

Painel, logs, capturas, exportação e ADB foram usados como evidências qualitativas de rastreabilidade.

A monografia não apresenta benchmark estatístico nem cálculo formal de erro em relação a ferramentas externas.

Arquitetura em camadas



Por que isso importa

Separar as camadas facilitou evoluir CPU, bateria, exportação e testes sem misturar responsabilidades.

Métricas e módulos implementados

Sinais coletados

FPS e *jank*
Rede HTTP
Bateria
rebuilds

Memória e heap
CPU do processo
Eventos
Tempos de frame

Módulos centrais

- FrameDataSource, MemoryDataSource e TelemetryHttpClient.
- Analyzer e Recommender.
- ExportService, CollectorBloc e ResourceCollector.

telemetria integrada no app

Dashboard integrado



Estado estável



Gargalos detectados

O painel agrega métricas e recomendações para orientar a primeira investigação técnica.

Publicação no pub.dev

Disponível para a comunidade

O pacote foi publicado no repositório oficial do ecossistema Dart/Flutter, permitindo instalação direta por outros desenvolvedores e pesquisadores.

Endereço

`pub.dev/packages/collector_flutter`

versão 0.1.2

Impacto prático

A publicação reforça reprodutibilidade, acesso aberto e possibilidade de evolução pela comunidade.

```
flutter pub add collector_flutter
```

Cenários controlados

1. Rebuilds	2. Rede	3. Memória	4. Combinado
Reconstruções intensivas de interface.	Requisições HTTP simultâneas.	Alocação intensiva de objetos.	Interface, rede, memória e CPU.

Cada cenário foi usado para verificar se os sinais técnicos acompanhavam a carga induzida.

Resultados observados

Cenário	Sinais registrados	Leitura para a defesa
Rebuilds	30 <i>rebuilds</i> , FPS 55,4, P95 20,4 ms e 2 frames longos.	A carga de interface foi registrada pelo painel.
Rede	9 requisições, FPS 42,9, P95 139,5 ms e 4 frames longos.	O cliente instrumentado registrou eventos e latências.
Memória	ADB: RSS 236.480 KB e PSS 222.232 KB.	Houve aumento real de memória; o snapshot do painel mostrou limite de leitura interna.
Combinado	17 requisições, 54 <i>rebuilds</i> , FPS 44,6, P95 37,7 ms e CPU 75,3%.	Múltiplos sinais apareceram juntos e geraram recomendações compatíveis.

Os valores vêm dos registros documentados na monografia: painel, logs, capturas e leituras ADB.

Evidências da validação funcional



Rebuilds



Rede



Memória



Carga combinada

Painel e capturas

Comportamento visual preservado no apêndice.

Logs e exportação

Valores e sessões rastreáveis fora do app.

ADB e testes

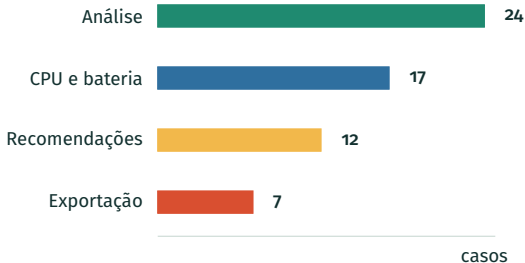
Apoio qualitativo e contratos internos verificados.

Testes automatizados

60 testes

Papel dos testes

Eles verificam cálculos, serialização, recomendações, exportação e *fallbacks* dos canais nativos.



Os testes complementam os cenários manuais: eles não substituem experimento quantitativo, mas dão confiança nos contratos internos.

Conclusão

- A biblioteca desenvolvida aproxima o primeiro diagnóstico de desempenho do fluxo real de desenvolvimento Flutter.
- A arquitetura em camadas sustentou coleta, análise, dashboard, exportação, CPU, bateria e testes.
- A avaliação foi funcional e qualitativa: cenários controlados, painel, logs, capturas, ADB e 60 testes.
- A publicação no pub.dev amplia o acesso ao pacote e permite que a comunidade teste, avalie e evolua a ferramenta.

Mensagem final

Performance não é apenas fazer o app rodar mais rápido; é fazer o app respeitar melhor o dispositivo, a bateria e a pessoa que o usa.

Perguntas?

O pacote `collector_flutter`

UF *m* G



Beatriz Vocurca Frade | Engenharia de Sistemas – UFMG